Save Point

Spaced-Repetition Scheduling Algorithm

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 23 October 2026

Implemented in `lib/core/scheduling.dart`, a pure function with no I/O, no side effects, no dependency on anything else in the app.

The interval sequence

```
const intervalSequenceDays = [1, 2, 3, 5, 7, 14, 30];
```

Every question tracks its current position as `intervalIndex` (0 to 6) on the `questions` table.

- **New question**: starts at `intervalIndex = 0`, first due one day after the note was saved (i.e. `next_due_at = created_at + 1 day`). This is a deliberate difference from the earlier Python desktop app, where a new question was due immediately; here, a fresh question always gets at least a day before its first review.
- **Correct answer**: advance one index (`currentIndex + 1`). Once past index 6 (the last index, 30 days), the sequence loops all the way back to index 0 and starts the full climb again, forever. This is also a deliberate difference from the Python app, which loops back to a mid-sequence "steady state" index rather than the very start; this project's version is simpler to state and reason about: right answer moves forward one step, and the climb from 1 day to 30 days always repeats identically once completed.
- **Incorrect answer**: reset immediately to `intervalIndex = 0`, regardless of what the current index was. The question re-climbs the full sequence from scratch.

Worked examples

Current index	Days	Correct?	New index	New days
0	1	yes	1	2
4	7	yes	5	14
6	30	yes	0	1 (loops back to the start)
3	5	no	0	1 (full reset)

API

```
int nextIntervalIndex(int currentIndex, bool wasCorrect)
int intervalDaysForIndex(int index)
DateTime computeNextDueAt(DateTime answeredAt, int newIndex)
ReviewOutcome applyReview(int currentIndex, bool wasCorrect, DateTime answeredAt)
bool isDue(DateTime nextDueAt, DateTime now)
```

`QuizScreen._submit()` is the only caller: it grades the answer, calls `applyReview()`, persists the result via `QuestionsRepo.updateScheduling()`, and records the full before/after transition in `question_history` via `ReviewsRepo.record()`. History is kept forever even though only the current `intervalIndex` drives scheduling going forward.

Review scope

`QuestionsRepo.listDue()` returns everything due across every folder, mixed together, not scoped to a single folder. There is currently no per-folder review filter (the Python desktop app has an optional one for its Stats screen; this app doesn't have a stats screen yet, see `future_work.md`).